

Introduction to Haskell Programming

Lists

S P Suresh

Chennai Mathematical Institute

Haskell @ CMI

Lists

- To describe a collection of values

Lists

- To describe a collection of values
- `[1,2,3,1]` is a list of `Int`

Lists

- To describe a collection of values
- `[1,2,3,1]` is a list of `Int`
- `[True,False,True]` is a list of `Bool`

Lists

- To describe a collection of values
- `[1,2,3,1]` is a list of `Int`
- `[True,False,True]` is a list of `Bool`
- Elements of a list must be of a uniform type

Lists

- To describe a collection of values
- `[1,2,3,1]` is a list of `Int`
- `[True,False,True]` is a list of `Bool`
- Elements of a list must be of a uniform type
- Cannot write `[1,2,True]` or `[3,'a']`

Lists

- List with values of type T has type $[T]$

Lists

- List with values of type `T` has type `[T]`
- `[1,2,3,1] :: [Int]`

Lists

- List with values of type `T` has type `[T]`
- `[1,2,3,1] :: [Int]`
- `[True,False,True] :: [Bool]`

Lists

- List with values of type `T` has type `[T]`
- `[1,2,3,1] :: [Int]`
- `[True,False,True] :: [Bool]`
- `[]` denotes the empty list, for all types

Lists

- List with values of type `T` has type `[T]`
- `[1,2,3,1] :: [Int]`
- `[True,False,True] :: [Bool]`
- `[]` denotes the empty list, for all types
- Lists can be nested

Lists

- List with values of type `T` has type `[T]`
- `[1,2,3,1] :: [Int]`
- `[True,False,True] :: [Bool]`
- `[]` denotes the empty list, for all types
- Lists can be nested
- `[[3,2], [], [7,7,7]] :: [[Int]]`

Internal representation

- To build a list, add one element at a time to the front (left)

Internal representation

- To build a list, add one element at a time to the front (left)
- Operator to append an element is :

Internal representation

- To build a list, add one element at a time to the front (left)
- Operator to append an element is `:`
- `1:[2,3] ---> [1,2,3]`

Internal representation

- To build a list, add one element at a time to the front (left)
- Operator to append an element is `:`
- `1:[2,3] ---> [1,2,3]`
- All Haskell lists are built this way, starting with `[]`

Internal representation

- To build a list, add one element at a time to the front (left)
- Operator to append an element is `:`
- `1:[2,3] ---> [1,2,3]`
- All Haskell lists are built this way, starting with `[]`
- `[1,2,3]` is actually `1:(2:(3:[]))`

Internal representation

- To build a list, add one element at a time to the front (left)
- Operator to append an element is `:`
- `1:[2,3] ---> [1,2,3]`
- All Haskell lists are built this way, starting with `[]`
- `[1,2,3]` is actually `1:(2:(3:[]))`
- `:` is right associative, so `1:2:3:[]` is `1:(2:(3:[]))`

Internal representation

- To build a list, add one element at a time to the front (left)
- Operator to append an element is `:`
- `1:[2,3] ---> [1,2,3]`
- All Haskell lists are built this way, starting with `[]`
- `[1,2,3]` is actually `1:(2:(3:[]))`
- `:` is right associative, so `1:2:3:[]` is `1:(2:(3:[]))`
- `1:[2,3] == 1:2:3:[]`, `1:2:[3] == [1,2,3]`, ...all return **True**

Decomposing lists

- Functions **head** and **tail**

Decomposing lists

- Functions **head** and **tail**
- **head** $(x:xs) \rightarrow x$

Decomposing lists

- Functions **head** and **tail**
- **head** $(x:xs) \rightarrow x$
- **tail** $(x:xs) \rightarrow xs$

Decomposing lists

- Functions **head** and **tail**
- **head** $(x:xs) \rightarrow x$
- **tail** $(x:xs) \rightarrow xs$
- Both undefined for `[]`

Decomposing lists

- Functions **head** and **tail**
- **head** $(x:xs) \rightarrow x$
- **tail** $(x:xs) \rightarrow xs$
- Both undefined for `[]`
- **Note:** **head** returns a value, **tail** returns a list

Decomposing lists

- Functions **head** and **tail**
- **head** $(x:xs) \rightarrow x$
- **tail** $(x:xs) \rightarrow xs$
- Both undefined for $[]$
- **Note:** **head** returns a value, **tail** returns a list
- **null** l is **True** exactly when l is $[]$

Defining functions on lists

- Recall inductive definition of numeric functions

Defining functions on lists

- Recall inductive definition of numeric functions
- Base case is $f\ 0$

Defining functions on lists

- Recall inductive definition of numeric functions
- Base case is $f\ 0$
- Define $f\ n$ in terms of n and $f\ (n-1)$

Defining functions on lists

- Recall inductive definition of numeric functions
- Base case is $f\ 0$
- Define $f\ n$ in terms of n and $f\ (n-1)$
- For lists, induction on list structure

Defining functions on lists

- Recall inductive definition of numeric functions
- Base case is $f\ 0$
- Define $f\ n$ in terms of n and $f\ (n-1)$
- For lists, induction on list structure
- Base case is the empty list

Defining functions on lists

- Recall inductive definition of numeric functions
- Base case is $f\ 0$
- Define $f\ n$ in terms of n and $f\ (n-1)$
- For lists, induction on list structure
- Base case is the empty list
- For a non-empty list $l = x:xs$

Defining functions on lists

- Recall inductive definition of numeric functions
- Base case is $f\ 0$
- Define $f\ n$ in terms of n and $f\ (n-1)$
- For lists, induction on list structure
- Base case is the empty list
- For a non-empty list $l = x:xs$
 - define $f\ l$ in terms of x and $f\ xs$

Examples

- Increment every element in an integer list

```
addOne :: [Integer] → [Integer]
addOne l = if null l
           then []
           else (head l + 1) : addOne (tail l)
```

Examples

- Increment every element in an integer list

```
addOne :: [Integer] → [Integer]
addOne l = if null l
  then []
  else (head l + 1) : addOne (tail l)
```

- Haskell will complain because **head** and **tail** are undefined on `[]`

Examples

- Increment every element in an integer list

```
addOne :: [Integer] → [Integer]
addOne l = if null l
  then []
  else (head l + 1) : addOne (tail l)
```

- Haskell will complain because **head** and **tail** are undefined on []
- Use patterns instead

```
addOne :: [Integer] → [Integer]
addOne l = if null l
  then []
  else let {x:xs = l} in (x+1) : addOne xs
```

Examples

- Compute the length of a list

```
myLength :: [Integer] → Integer
myLength l = if null l
  then 0
  else 1 + myLength (tail l)
```

Examples

- Compute the length of a list

```
myLength :: [Integer] → Integer
myLength l = if null l
  then 0
  else 1 + myLength (tail l)
```

- Once again, Haskell will complain, so use patterns instead

```
myLength :: [Integer] → Integer
myLength l = if null l
  then 0
  else let {_:xs = l} in 1 + myLength xs
```

Pattern matching

- `[]` is the pattern that matches the empty list

Pattern matching

- `[]` is the pattern that matches the empty list
- A nonempty list decomposes uniquely as `x:xs`

Pattern matching

- `[]` is the pattern that matches the empty list
- A nonempty list decomposes uniquely as `x:xs`
- Pattern matching implicitly separates head and tail

Pattern matching

- `[]` is the pattern that matches the empty list
- A nonempty list decomposes uniquely as `x:xs`
- Pattern matching implicitly separates head and tail
- Empty list will not match this pattern `x:xs`

Pattern matching

- `[]` is the pattern that matches the empty list
- A nonempty list decomposes uniquely as `x:xs`
- Pattern matching implicitly separates head and tail
- Empty list will not match this pattern `x:xs`
- We should enclose the nonempty list pattern with parentheses: `(x:xs)`

```
myLength :: [Integer] → Integer
myLength []          = 0
myLength (x:xs)      = 1 + myLength xs
```

Pattern matching

- `[]` is the pattern that matches the empty list
- A nonempty list decomposes uniquely as `x:xs`
- Pattern matching implicitly separates head and tail
- Empty list will not match this pattern `x:xs`
- We should enclose the nonempty list pattern with parentheses: `(x:xs)`

```
myLength :: [Integer] → Integer
myLength []          = 0
myLength (x:xs)      = 1 + myLength xs
```

- Built-in function `length`

Examples

- `addAtEnd x l` adds `x` at the end of `l`

```
addAtEnd :: Int → [Int] → [Int]
```

```
addAtEnd x [] = [x]
```

```
addAtEnd x (y:ys) = y:addAtEnd x ys
```

Examples

- `addAtEnd x l` adds `x` at the end of `l`

```
addAtEnd :: Int → [Int] → [Int]
addAtEnd x []          = [x]
addAtEnd x (y:ys)     = y:addAtEnd x ys
```

- `attach l1 l2` attaches `l2` to the end of `l1`

```
attach :: [Int] → [Int] → [Int]
attach l1 []      = l1
attach l1 (y:ys)  = attach (addAtEnd l1 y) ys
```

Examples

- `attach l1 l2` requires more than `length l1 * length l2` steps

Examples

- `attach l1 l2` requires more than `length l1 * length l2` steps
- Smarter version recurses on the first list:

```
attach :: [Int] → [Int] → [Int]
attach [] l2      = l2
attach (x:xs) l2  = x:attach xs l2
```

Examples

- `attach l1 l2` requires more than `length l1 * length l2` steps
- Smarter version recurses on the first list:

```
attach :: [Int] → [Int] → [Int]
attach [] l2      = l2
attach (x:xs) l2  = x:attach xs l2
```

- This takes around `length l1` steps

Examples

- `attach l1 l2` requires more than `length l1 * length l2` steps
- Smarter version recurses on the first list:

```
attach :: [Int] → [Int] → [Int]
attach [] l2      = l2
attach (x:xs) l2  = x:attach xs l2
```

- This takes around `length l1` steps
- Built-in function `++`

Examples

- `attach l1 l2` requires more than `length l1 * length l2` steps
- Smarter version recurses on the first list:

```
attach :: [Int] → [Int] → [Int]
attach [] l2      = l2
attach (x:xs) l2  = x:attach xs l2
```

- This takes around `length l1` steps
- Built-in function `++`
- `[3,2,4] ++ [5,7,6]` is `[3,2,4,5,7,6]`

Example: valueAtPosition

- Handling the problem cases:

```
valueAtPosition n _  
  | n < 0          = error "Negative index"  
valueAtPosition _ [] = error "Index too large"  
valueAtPosition 0 (x:xs) = x  
valueAtPosition n (x:xs) = valueAtPosition (n-1) xs
```

Example: valueAtPosition

- Handling the problem cases:

```
valueAtPosition n _  
  | n < 0          = error "Negative index"  
valueAtPosition _ [] = error "Index too large"  
valueAtPosition 0 (x:xs) = x  
valueAtPosition n (x:xs) = valueAtPosition (n-1) xs
```

- If $n \geq \text{length } l$, we will eventually hit a recursive call where $n \geq 0$ and the list is `[]`

Example: valueAtPosition

- Handling the problem cases:

```
valueAtPosition n _  
  | n < 0          = error "Negative index"  
valueAtPosition _ [] = error "Index too large"  
valueAtPosition 0 (x:xs) = x  
valueAtPosition n (x:xs) = valueAtPosition (n-1) xs
```

- If $n \geq \text{length } l$, we will eventually hit a recursive call where $n \geq 0$ and the list is `[]`
- error** prints an error message and aborts (matches any type)

Reversing a list

- Remove the head

Reversing a list

- Remove the head
- Recursively reverse the tail

Reversing a list

- Remove the head
- Recursively reverse the tail
- Add head at end

```
myReverse :: [Int] → [Int]
myReverse []      = []
myReverse (x:xs) = myReverse xs ++ [x]
```

Reversing a list

- Remove the head
- Recursively reverse the tail
- Add head at end

```
myReverse :: [Int] → [Int]
myReverse []      = []
myReverse (x:xs) = myReverse xs ++ [x]
```

- Number of steps is proportional to n^2 , where n is the length

Reversing a list

- Remove the head
- Recursively reverse the tail
- Add head at end

```
myReverse :: [Int] → [Int]
myReverse []      = []
myReverse (x:xs) = myReverse xs ++ [x]
```

- Number of steps is proportional to n^2 , where n is the length
- Built-in function **reverse** is smarter

Built-in function: **reverse**

- The built-in **reverse** takes time proportional to n , the length of the list

Built-in function: **reverse**

- The built-in **reverse** takes time proportional to n , the length of the list
- **Strategy**: Repeatedly extract head and place it in front of an accumulator list

Built-in function: **reverse**

- The built-in **reverse** takes time proportional to n , the length of the list
- **Strategy**: Repeatedly extract head and place it in front of an accumulator list
- The list is automatically reversed

```
reverse l      = revInto [] l where
  revInto a [] = a
  revInto a (x:xs) = revInto (x:a) xs
```